

# 实验报告：性能分析与优化

| 项目   | 内容                      |
|------|-------------------------|
| 课程名称 | 系统开发工具基础                |
| 实验主题 | Profiling / Performance |
| 题目编号 | 第8题                     |
| 学号   | 25020007191             |
| 姓名   | 周御基                     |
| 实验日期 | 2026年8月31日              |

## 一、实验目的

1. 编写慢速词频统计程序，并生成用于测试的单词列表
2. 使用 `python -m cProfile -s cumulative` 定位性能瓶颈
3. 优化慢速程序，保证输出结果与原程序一致
4. 分别计算中位数运行时间并计算加速比

## 二、实验环境

- 操作系统：Ubuntu Linux（远程服务器 192.168.79.130）
- Python：3.13.3
- 工具：cProfile（内置）、time、自研bench基准脚本
- 远程连接：SSH（paramiko）

## 三、实验步骤与代码

### 步骤1：生成用于测试的单词列表

创建 `generate_words.py`：从 5000 个互异单词的词汇表随机抽取 20 万次，写入 `words.txt`。

```
import random

VOCAB = [f"w{i:04d}" for i in range(5000)]

def generate_words(n):
    return [random.choice(VOCAB) for _ in range(n)]

if __name__ == "__main__":
    random.seed(42)
    words = generate_words(200_000)
    with open("words.txt", "w") as f:
        f.write("\n".join(words))
    print("wrote", len(words), "words from vocab", len(VOCAB))
```

```
python3 generate_words.py
```

**输出结果：**

```
wrote 200000 words from vocab 5000
```

## 步骤2：编写慢速词频统计程序

慢速版 wordfreq.py：用「词频表列表 + 线性扫描」计数。每个单词都要线性扫描已统计的词频表寻找，时间复杂度为  $O(n)$ 。这是典型的低效实现。

```

def _find(freqs, word):
    """线性扫描词频表，返回索引；未找到返回 -1 ( $O(n)$ ) """
    for idx, (w, c) in enumerate(freqs):
        if w == word:
            return idx
    return -1

def word_frequency(fname):
    freqs = []
    for line in open(fname):
        word = line.strip()
        idx = _find(freqs, word)
        if idx ≥ 0:
            freqs[idx] = (freqs[idx][0], freqs[idx][1] + 1)
        else:
            freqs.append((word, 1))
    return freqs

def top_words(freqs, k=5):
    items = sorted(freqs, key=lambda kv: kv[1], reverse=True)
    return items[:k]

if __name__ == "__main__":
    freqs = word_frequency("words.txt")
    print("unique words:", len(freqs))
    print("top:", top_words(freqs, 5))

```

运行（基准时间）：

```
time python3 wordfreq.py
```

```

unique words: 5000
top: [('w4265', 66), ('w3531', 65), ('w4813', 62), ('w2965', 61), ('w3460', 61)]
real    0m18.905s

```

慢速版约需 18.9 秒。

### 步骤3：使用 cProfile 定位瓶颈

```
python3 -m cProfile -s cumulative wordfreq.py
```

## 输出结果 (cumulative 排序顶部) :

410310 function calls in 31.116 seconds

Ordered by: cumulative time

| ncalls | totttime | percall | cumtime | percall | filename:lineno(function)         |
|--------|----------|---------|---------|---------|-----------------------------------|
| 1      | 0.000    | 0.000   | 31.116  | 31.116  | {built-in method builtins.exec}   |
| 1      | 0.000    | 0.000   | 31.116  | 31.116  | wordfreq.py:1(<module>)           |
| 1      | 6.164    | 6.164   | 30.976  | 30.976  | wordfreq.py:8(word_frequency)     |
| 200000 | 21.805   | 0.000   | 21.805  | 0.000   | wordfreq.py:1(_find)              |
| 200000 | 2.927    | 0.000   | 2.927   | 0.000   | {method 'strip' of 'str' objects} |
| 1      | 0.000    | 0.000   | 0.140   | 0.140   | wordfreq.py:19(top_words)         |
| 1      | 0.071    | 0.071   | 0.140   | 0.140   | {built-in method builtins.sorted} |

## 瓶颈分析:

- `_find` 被调用了 **200000 次**，累计耗时 **21.805s** (占绝大部分)。
- 每次 `_find` 都是对不断增长的词频表做线性扫描 ( $O(n)$ )，200000 个单词叠加导致总复杂度约  $O(n^2)$ ，成为绝对热点。
- 其次是 `str.strip()` (2.927s)，属必要的输入整理，非主因。

## 步骤4: 优化慢速程序

优化版 `wordfreq_fast.py`：用 Python 字典 (哈希表) 以  $O(1)$  完成查找与计数，替代  $O(n)$  线性扫描；`dict.get` 一行即完成「存在则 +1，否则置 1」。

```
def word_frequency_fast(fname):
    """优化版：用字典 O(1) 计数，替代 O(n) 线性扫描"""
    freqs = {}
    for line in open(fname):
        word = line.strip()
        freqs[word] = freqs.get(word, 0) + 1
    return freqs

def top_words(freqs, k=5):
    items = sorted(freqs.items(), key=lambda kv: kv[1], reverse=True)
    return items[:k]

if __name__ == "__main__":
    import sys
    freqs = word_frequency_fast("words.txt")
    print("unique words:", len(freqs))
    print("top:", top_words(freqs, 5))
```

**优化点：**把“列表线性查找”替换为“哈希表直接寻址”，查找从  $O(n)$  降为  $O(1)$ ，整体从  $O(n^2)$  降为  $O(n)$ 。

```
time python3 wordfreq_fast.py
```

```
unique words: 5000
top: [('w4265', 66), ('w3531', 65), ('w4813', 62), ('w2965', 61), ('w3460', 61)]
real    0m0.075s
```

结果与原程序**完全一致**（同一份 top 5），证明优化未改变语义。

## 步骤5：分别计算中位数运行时间与加速比

编写 `bench.py`，各自独立运行 5 次、计时，取中位数（排除首尾波动影响）：

```

import subprocess
import time
import sys

def run(file):
    t0 = time.perf_counter()
    subprocess.run([sys.executable, file], check=True)
    return time.perf_counter() - t0

def median(xs):
    s = sorted(xs); n = len(s)
    return s[n // 2] if n % 2 else (s[n // 2 - 1] + s[n // 2]) / 2

slow_times = [run("wordfreq.py") for _ in range(5)]
fast_times = [run("wordfreq_fast.py") for _ in range(5)]
slow_med = median(slow_times); fast_med = median(fast_times)
print("slow raw:", [f"{t:.3f}" for t in slow_times])
print("fast raw:", [f"{t:.3f}" for t in fast_times])
print(f"SLOW median = {slow_med:.3f}s")
print(f"FAST median = {fast_med:.3f}s")
print(f"speedup ratio = {slow_med / fast_med:.1f}x")

```

```
python3 bench.py
```

## 输出结果:

```

slow version raw: ['19.386', '18.995', '18.783', '19.343', '18.627']
fast version raw: ['0.075', '0.080', '0.080', '0.090', '0.086']
SLOW median = 18.995s
FAST median = 0.080s
speedup ratio = 237.1x

```

# 四、实验结果

## 4.1 结果展示

| 指标      | 慢速版            | 优化版          |
|---------|----------------|--------------|
| 中位数运行时间 | 18.995s        | 0.080s       |
| 单次最慢/最快 | 19.386/18.627s | 0.090/0.075s |

| 指标      | 慢速版 | 优化版            |
|---------|-----|----------------|
| 输出 top5 | 一致  | 一致             |
| 加速比     | —   | <b>237.1 ×</b> |

## 4.2 结果分析

1. **瓶颈定位准确**：cProfile 的 `-s cumulative` 显示 `_find` 累计 21.8s 成为热点；20 万次调用对应每个单词一次线性查找，复杂度  $O(n^2)$ 。
2. **优化手段针对性**：用字典哈希表把查找从  $O(n)$  降到  $O(1)$ ，属算法复杂度优化而非微调，因而收益巨大。
3. **正确性保持**：优化前后的 top5 与 unique words 完全一致，未改变统计结果。
4. **加速比**：优化版中位数 0.080s  $\approx$  慢速版 18.995s 的  $1/237$ ，加速比达到 **237.1 ×**。
5. 中位数相比均值更能反映典型性能，避免因系统抖动偏向极端值。

## 五、实验总结

通过本次实验，我掌握了以下技能：

1. 用 `python -m cProfile -s cumulative` 对程序进行性能剖析并定位热点
2. 从复杂度角度分析瓶颈，将  $O(n)$  线性查找优化为  $O(1)$  哈希查找
3. 通过多次计时取中位数评估性能，并计算加速比
4. 理解“先测量、后优化、再验证正确性”的性能优化流程

实验中的脚本文件：

1. `Scripts/q08/generate_words.py`
2. `Scripts/q08/wordfreq.py`（慢速版）
3. `Scripts/q08/wordfreq_fast.py`（优化版）
4. `Scripts/q08/bench.py`